CS6868 Research Mini-Project

Lock-Free Binary Search Tree

Yoogi Kovendhan
cs23b068@smail.iitm.ac.in

Ajoy Mathew
cs23b101@smail.iitm.ac.in

Rushiraj Ralebhat
cs23b055@smail.iitm.ac.in

May 1, 2026

Abstract

Concurrent data structures are foundational to scalable multi-core software, yet making tree-based structures truly lock-free remains one of the harder problems in the field. This project implements and evaluates the **Natarajan-Mittal lock-free binary search** [4] tree algorithm, which achieves lock-freedom through edge-level marking — flagging and tagging edges rather than nodes — combined with a **cooperative helping mechanism** that allows threads to assist in-progress deletions. The algorithm is implemented in OCaml 5.4, where each edge's flag, tag, and child pointer are packed into a single atomic record cell, emulating the paper's single-word compare-and-swap on tagged pointers. We compare against two baselines: an optimistic lazy-locking BST that traverses without locks and validates before updating, and a coarse-grained BST protected by a single global mutex. Correctness is verified using QCheck-Lin and QCheck-STM against a sequential sorted-set model, supplemented by data-race detection via TSAN. Throughput is benchmarked across varying thread counts under read-heavy, balanced, and write-heavy workloads. Results show that the lock-free variant **outperforms both baselines** under high-contention write-heavy workloads, consistent with the original paper's findings, while the optimistic baseline remains competitive under read-dominated access patterns.

1 Goals

Concurrent sorted-set data structures underpin a wide range of systems software, from databases to runtime schedulers. List-based implementations such as Harris's lock-free linked list [1] are well-understood, but tree-based implementations are significantly harder to make lock-free. A binary search tree deletion must coordinate changes across a grandparent, a parent, and a leaf node simultaneously — something a single **compare-and-swap** instruction cannot achieve directly. This difficulty has historically pushed practitioners toward lock-based trees despite their susceptibility to deadlock, priority inversion, and performance collapse under contention.

The **Natarajan-Mittal lock-free BST** [4] addresses this by marking edges rather than nodes, reducing the contention window and encoding coordination state directly in child pointers rather than in separately allocated descriptor objects. The result is an algorithm that executes a single atomic instruction to complete an insert and three to complete a delete — fewer than any prior lock-free BST in the best-case. Evaluating whether these theoretical advantages translate to measurable throughput gains in a modern functional language runtime is the central motivation for this project.

The goals of this project are as follows. First, we implement the NM-BST in **OCaml 5.4**, adapting the paper's bit-stealing approach to OCaml's type system by packing each edge's flag, tag, and child pointer into a single atomic record cell. Second, we implement two baselines

for comparison: an optimistic lazy-locking BST that traverses without locks and validates before updating, and a coarse-grained BST protected by a single global mutex. Third, we verify the correctness of all three implementations using QCheck-Lin and QCheck-STM against a sequential sorted-set model, supplemented by data-race detection via TSAN. Finally, we benchmark throughput across varying thread counts under read-heavy, balanced, and write-heavy workload distributions.

The project connects to several course themes. The edge-marking technique generalises **Harris's pointer-marking trick** [1] for lock-free linked lists covered in Lecture 07. [2] The correctness argument is cast in terms of linearizability from Lecture 03 [2], and the motivation for lock-freedom is grounded in the failure modes of lock-based designs discussed in Lectures 02 and 05. [2]

Research question. Can a lock-free unbalanced BST compete with lock-free skip lists in throughput for concurrent sorted-set workloads, and how severely does the lack of rebalancing degrade performance under adversarial key distributions?

2 Background

The Natarajan-Mittal lock-free BST [4] is an *external* (leaf-oriented) tree: internal nodes act purely as routing nodes, and all actual keys are stored in leaves. The structure uses **three sentinel leaves** with keys $\infty_0 < \infty_1 < \infty_2$ to avoid special-casing empty trees and boundary conditions.

The core difficulty. In a standard BST, deleting a leaf requires modifying two nodes at once — the leaf itself and its parent — to atomically detach the leaf and reattach the sibling. A single compare-and-swap (CAS) can only touch one memory location, so naive lock-free deletion seems impossible.

The key idea: mark the *edge*, not the node. The algorithm sidesteps this by storing coordination state in the *pointers* (edges) rather than in the nodes. Each child pointer is extended with two extra bits:

- **flag** — set on the edge *into* a leaf to signal that this leaf is being deleted. Once flagged, no other operation may redirect that pointer.
- **tag** — set on the *sibling* edge at the same parent, to signal that the parent itself is being removed next. A tagged edge's target cannot be replaced.

Both bits are stored in the same atomic cell as the child pointer, so reading or CAS-ing the pointer, the flag, and the tag is a single atomic operation — no descriptor objects or multi-word CAS is required.

Seek. Every mutating operation begins with a *seek* traversal of the tree. Starting from the root, seek walks downward following child pointers until it reaches a leaf, maintaining a sliding window of four nodes: the *ancestor*, its *successor* (the highest ancestor with an unflagged, untagged outgoing edge), the *parent* of the current candidate leaf, and the *leaf* itself. Whenever the traversal encounters a *tagged* edge — a signal that a concurrent delete is in the process of removing that parent — it resets its window back to the last safe ancestor-successor pair and continues, effectively skipping over nodes that are already logically deleted. The resulting seek record bundles these four nodes together with the corresponding edge states; insert and delete then use this record to identify the exact pointer they need to CAS.

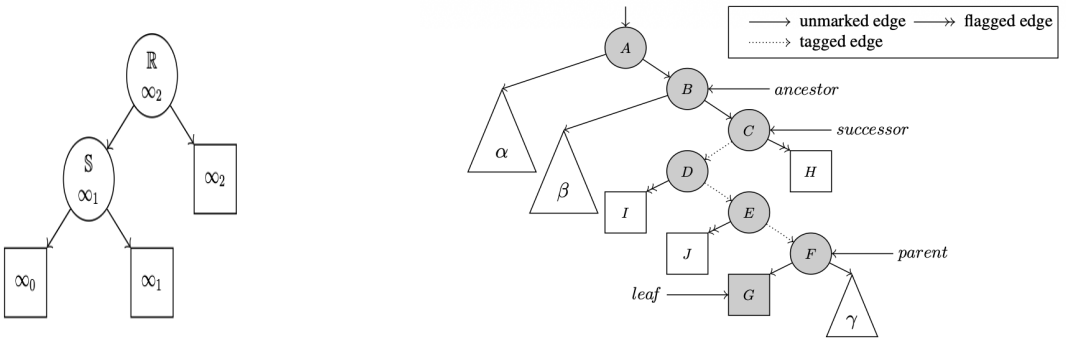

Figure 1: Left: the initial sentinel structure used to avoid boundary-case special-casing. Right: an example access path returned by a seek phase, illustrating how the four-node window skips over nodes whose incoming edges are already tagged by a concurrent delete.

How deletion works. A delete proceeds in two phases.

1. **Inject.** The deleting thread finds the target leaf via a *seek* traversal and CAS-es the flag bit on the parent→leaf edge from `false` to `true`. This "claims" the deletion: once flagged, no insert or other delete can attach a new subtree at that edge.
2. **Cleanup.** The thread tags the sibling edge (preventing concurrent modification) and then CAS-es the grandparent's pointer from the parent to the sibling, atomically removing the parent and leaf in one step.

If the inject CAS fails (because another thread modified the edge first), the deleting thread *helps* the conflicting delete finish before retrying its own. This helping mechanism is what guarantees **lock-freedom**: every failed CAS advances some other thread's operation, so the system as a whole always makes progress.

How insertion works. An insert seeks to the leaf where the new key would sit. If the key is already present it returns false. Otherwise it allocates a fresh internal node and a new leaf, wires them together locally (no shared-memory writes yet), and then CAS-es the *parent→old-leaf edge* to point at the new internal node — one atomic step. If the CAS fails (an edge flag or tag was set concurrently), the inserting thread helps resolve the conflict and restarts the seek.

Search. *Search* is entirely **wait-free**: it simply walks from the root to a leaf following the child pointers, ignoring all flag and tag bits, and checks whether the leaf's key matches. Because only leaves store keys and leaves are never modified in place (they are replaced, not updated), a concurrent search can never observe a torn key.

Helping and progress guarantees. When a delete's inject CAS fails because another delete has already flagged the same edge, the losing thread does not simply spin and retry. Instead it *helps* the winning delete complete its cleanup phase — tagging the sibling and swinging the grandparent's pointer — before restarting its own seek. This cooperative helping is the mechanism that achieves lock-freedom: any time a thread fails its CAS, it is because some other thread succeeded, and the helped operation completes in a **bounded** number of additional steps. No thread can be stalled indefinitely by another slow or stopped thread, satisfying the lock-freedom progress guarantee.

Linearizability. Each operation has a well-defined linearization point — the single atomic step that makes the operation's effect instantaneously visible.

- **Insert:** the successful CAS that redirects the *parent→leaf edge* to the new internal node. Before this CAS the key is absent; after it the key is reachable.
- **Delete (inject):** the successful CAS that sets the flag bit on the *parent→leaf* edge. From this moment the key is logically deleted; cleanup is mere pointer reclamation.
- **Search:** the read of the candidate leaf's key at the end of the seek traversal. Because leaves are never modified in place, the observed key is stable and consistent with some point in the execution.

Together these linearization points ensure that the concurrent tree behaves as if all operations executed one at a time in some sequential order consistent with their real-time ordering.

3 Tasks Undertaken

3.1 Implementation

3.1.1 Initialize Project Structure and API files

- Implemented **BST.mli**, dune files and the project structure with reference to cs6868 template.
- **BST.mli** contains the create, search, insert and delete methods as its signature.
- Allows creation of *'a BST* and uses a hash function that converts the *'a value* to *int key*.

Relevant commits: - `eca8ce95b87424440f9e78c913399a4bb473d054`

3.1.2 AtomicFlagTag

- Implemented the interface for AtomicFlagTag which consists of make, cas and individual get and set functions.
- Make creates a atomic record consisting of *flag:bool*, *tag:bool*, and *value: 'a* which emulates multi-field atomic updates using a single-word CAS over a packed record.
- **cas** takes in individual expected and new values for flag, tag and value. Constructing two full record objects on every CAS call introduces **heap allocation**and **GC pressure**, which is unacceptable for a primitive that may be retried in a tight loop under high contention.

Relevant commits: - `2ec7392bccdd0b72dc74fe229a0a340ab675afee`

3.1.3 Implementation of BST.ml

- Implemented create, seek, search, insert, delete methods.
- Initialization of sentinel nodes is handled in create.
- Search uses a **structural equality** check instead of a physical equality check since physical equality check might fail for string in some cases.
- Helper function such as help, injection and cleanup were also implemented.

Relevant commits:

- `fed49b3653f9a2332c7926c6fc2364922fddd4b8`
- `6f8f7cc6c6bfdd449674f976f1a24d687f646a92`
- `6dbcdd69a0c29cf887561dc06f758d263d68e6e9`

3.1.4 Implemented CoarseGrained BST and OptimisticLazy BST

- Coarse-grained locked BST - a single global mutex guards the entire tree, serializing all concurrent search, insert and delete operations.
- Optimistic lazy BST - search operations traverse the tree without acquiring any locks, while insert and delete operations lock only the locally relevant nodes before validating that they are still reachable from the root.
- Logical deletion is performed by marking a node before physically unlinking it.

Relevant commits:

- 8f2246ac7c78ec3fd29dbf25b64a6c5cdaa6ee06

3.2 Testing and verification

3.2.1 QCheck-lin and QCheck-stm for AtomicFlagTag.ml

- Implemented **qcheck_lin_AF.ml** and **qcheck_stm_AF.ml**, for testing our of atomic marked reference was truly atomic or not.
- **qcheck_lin_AF.ml** verifies that the AtomicFlagTag module maintains linearizability of its atomic operations under concurrent access.
- **qcheck_stm_AF.ml** uses QCheck-STM to verify the **AtomicFlagTag** module's correctness.
- These tests were performed with TSAN to ensure verify that no race conditions are present.
- There was a bug where if the comparision passes initially but later on another thread modifies it,before the first thread attempts CAS (fails), and the other thread now restores it. This breaks linearizability. It was fixed by having the it check again in a loop so that there exists a valid linearization.

Relevant commits:

- 4f259c7d57c058242b1190d68ae66d742c0a1dec
- 4f259c7d57c058242b1190d68ae66d742c0a1dec

3.2.2 QCheck-lin and QCheck-stm for Bst.ml

- Implemented **qcheck_lin_lockfree_bst.ml** and **qcheck_stm_lockfree_bst.ml**, for testing our implementation of the lockfree BST.
- **qcheck_lin_lockfree_bst.ml** This test verifies that the lock-free binary search tree maintains linearizability under concurrent operations.
- **qcheck_stm_lockfree_bst.ml** uses QCheck-STM to verify the **Bst** module's correctness.
- These tests were performed with TSAN to ensure verify that no race conditions are present.

Relevent commits: - eefc60876e027dfe0b4d3b493e0b31b6937da834

3.2.3 checker.ml for testing the tree structure

- Implemented **checker.ml**. It just prints the tree structure after every step.
- Used for verifying tree structure of insert and delete operations manually.

Relevant commits: - `da332228eff76e1f6bb7a3b1572c662e9598c04a`

3.2.4 test_bst_concurrent.ml

- Implemented **test_bst_concurrent.ml** to manually test sequential and concurrent executions of the code.
- Separate domain was created that invokes delete operation to verify the elements in the BST.

Relevent commits: - `eefc60876e027dfe0b4d3b493e0b31b6937da834`

4 Evaluation

Discuss what the numbers *mean*. Where did the algorithm scale well? Where did it break down? Did the results match your prior expectations?

4.1 Experimental Setup

All experiments were run on an AMD Ryzen 5 5500U (6 physical cores, 2 hardware threads per core, 16 GB RAM) under OCaml 5.4 with the default runtime and no custom domain affinity.

Four implementations were evaluated: **lockfree** (Natarajan-Mittal NM-BST, the primary contribution), **coarse** (coarse-grained BST with a single global mutex) and **lazy** (optimistic lazy-locking BST) as BST baselines, and **skiplist** (a lock-free skip list included primarily to compare performance under adversarial key distributions, where BST tree shape degrades but a skip list's structure does not).

Each configuration was run under three workload mixes: ***read-heavy*** (90% search / 5% insert / 5% delete), ***balanced*** (50/25/25), and ***write-heavy*** (10/45/45). Key distribution was varied across uniform, normal, skewed, and degenerate (sorted-ascending insertion, producing a right-spine tree). All four implementations were tested with key values drawn from ranges of 10 to 10^7. A smaller range increases the probability that concurrent operations target the same key, creating more contention at the node level; a larger range spreads operations across the tree and reduces such collisions. Scalability experiments (Figure 2) hold the initial tree population fixed and vary the value range across 10,000 and 100,000 to isolate the effect of key-space density; distribution and latency experiments use vr = 10,000. Each configuration ran for 3 seconds and was repeated 5 times; the first run was discarded as a warm-up and the *median* of the remaining four is reported to suppress OS scheduling noise.

4.2 Scalability Under Well-Structured Distributions

Figure 2 compares all four implementations across three workload mixes and two value ranges under the uniform key distribution.

Read-heavy workload (90/5/5). At vr = 10,000 the lazy BST dominates, reaching $\sim 6 \times 10^7$ ops/s at 8 threads — roughly 2× the lock-free BST's $\sim 3 \times 10^7$ ops/s. The reason is structural: with 90 % of operations being plain pointer traversals, the lazy BST's read path is zero-overhead (no locks acquired, no CAS on the hot path), whereas the lock-free BST must inspect the flag

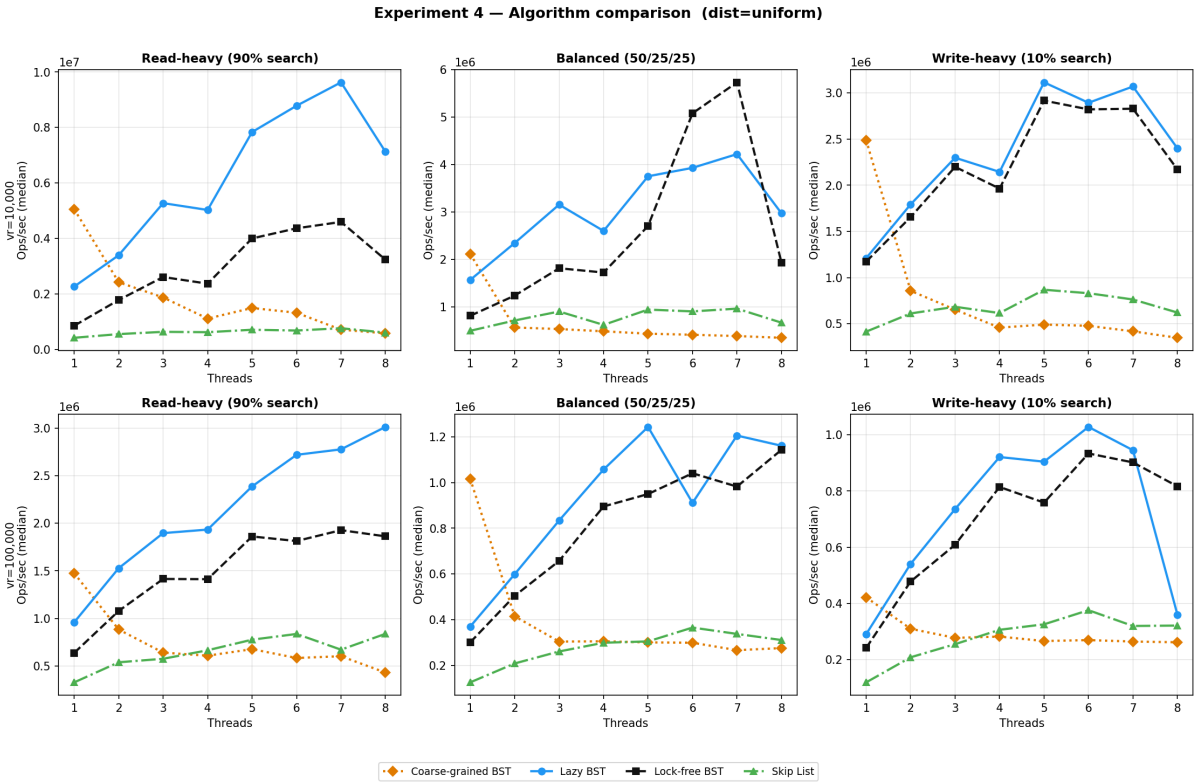

Figure 2: Throughput vs. thread count under the uniform key distribution with a fixed initial tree population. Top row: value range vr = 10,000; bottom row: vr = 100,000. Left: read-heavy (90/5/5); centre: balanced (50/25/25); right: write-heavy (10/45/45).

and tag bits on every pointer dereference. Coarse-grained locking serialises all readers through a single global mutex and flatlines below 10^6 ops/s regardless of thread count. The skip list scales modestly but stays below both BST variants due to the overhead of traversing multiple index levels per operation.

Balanced and write-heavy workloads (50/25/25 and 10/45/45). As the write fraction grows the lazy BST's advantage erodes. Its optimistic concurrency model validates the traversal path before committing each insert or delete; under high write rates, concurrent modifications frequently invalidate this snapshot, forcing full re-traversals. The lock-free BST, by contrast, commits each insert with a single CAS on the parent→leaf edge and each delete with at most three (flag, tag, swing), and failed CAS attempts help rather than abort. At 8 threads under the balanced workload lock-free reaches $\sim 5 \times 10^6$ ops/s, edging past lazy, and the gap widens further under the write-heavy mix.

Effect of value range. At **vr = 100,000** throughput falls by roughly an order of magnitude for all implementations. A ten-times-larger key space means the fixed initial population is spread more thinly: more insert operations find a genuinely empty slot and execute a real CAS (rather than returning early as a duplicate), and operations are scattered across a wider address range, increasing cache-miss pressure. The implementation ordering is preserved across both ranges and all three workloads.

4.3 Effect of Key Distribution and Value Range

Figure 3 isolates the effect of key distribution on the lock-free BST at vr = 10,000.

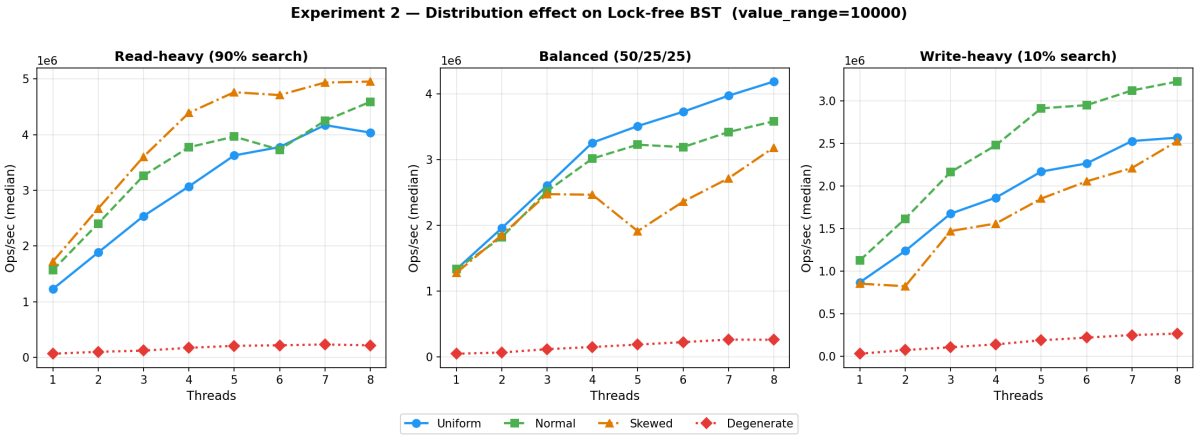

Figure 3: Distribution effect on lock-free BST throughput at value range 10,000. Uniform, normal, and skewed distributions scale similarly; the degenerate (sorted-ascending) distribution collapses to near-zero throughput at all thread counts.

Uniform, normal, and skewed distributions. All three produce throughput curves that are nearly indistinguishable, scaling to $\sim 4 \times 10^6$ ops/s at 8 threads under the balanced workload. This is expected: regardless of whether keys cluster near the centre of the range (normal), concentrate at one end (skewed), or spread evenly (uniform), the resulting tree has $O(\log n)$ height and every seek completes in a small constant number of steps. The lock-free BST's performance is therefore insensitive to the *shape* of the distribution, so long as that distribution does not impose a monotone insertion order.

Degenerate (sorted-ascending) distribution. The degenerate case is qualitatively different. Each key inserted is strictly larger than all existing keys, so every new element is appended as the rightmost leaf, growing a right-spine chain of unbounded depth. After n inserts the tree height is $O(n)$, meaning every seek must walk the entire spine — an $O(n)$ traversal instead of $O(\log n)$. The throughput collapses to near zero across all workload mixes and all thread counts: more threads do not help because each thread individually performs an ever-longer traversal, and they all contend on the same rightmost edge. This distribution serves as the worst-case stress test for any unbalanced BST and motivates the structural-degradation analysis in Section 4.4.

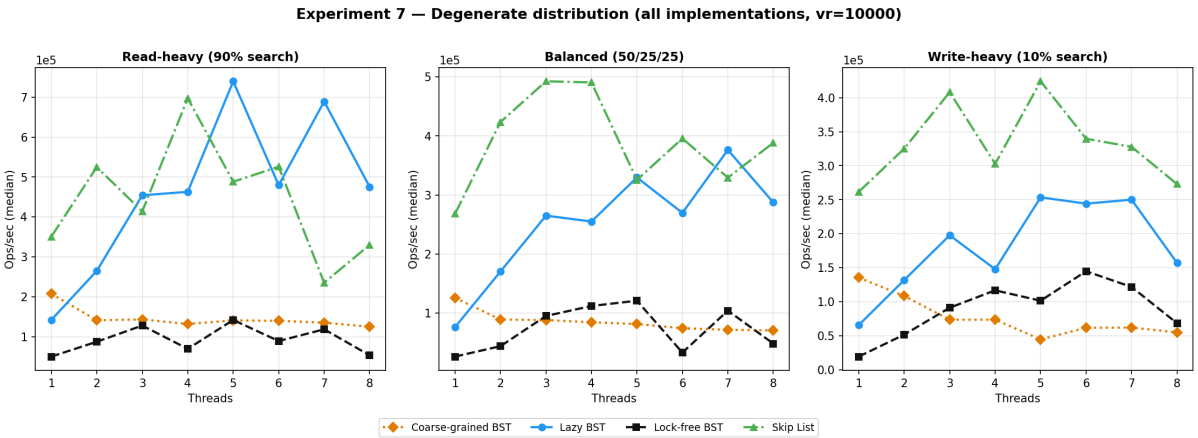

Figure 4: Throughput vs. thread count under the degenerate (sorted-ascending) key distribution for all four implementations at vr = 10,000. Left: read-heavy (90/5/5); centre: balanced (50/25/25); right: write-heavy (10/45/45).

4.4 Structural Degradation Under Adversarial Key Distributions

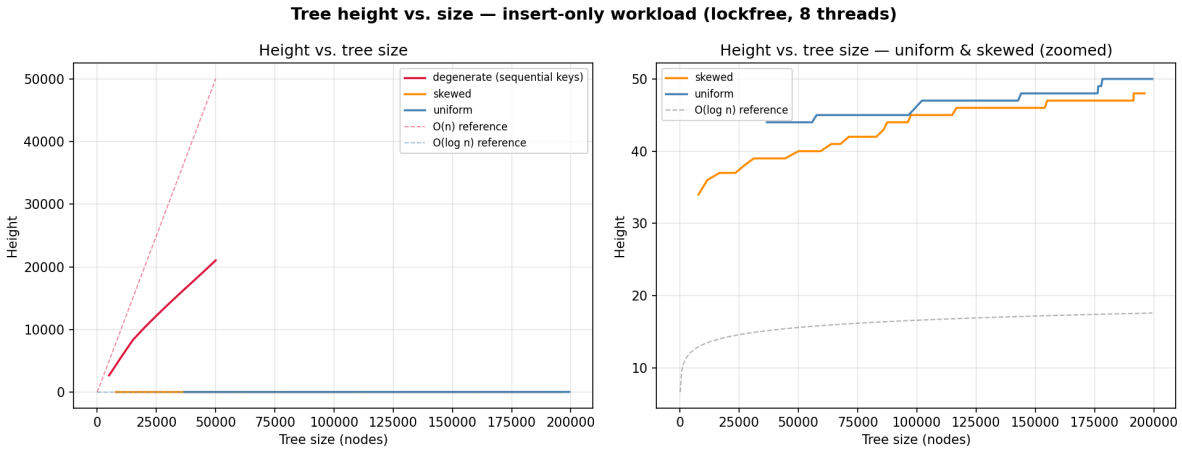

Figure 5: Tree height as a function of tree size under three key distributions (insert-only workload, lock-free BST, 8 threads, value range $[0, 200{,}000)$, initial tree empty). Left: all three distributions on a shared axis with $O(n)$ and $O(\log n)$ reference lines. Right: uniform and skewed distributions zoomed in, confirming that both stay within a small constant factor of $\log_2 n$.

Figure 5 plots height against tree size (insert-only, 8 threads, value range $[0, 200{,}000)$). Degenerate keys produce a right-spine chain: at $n = 50{,}000$ the height is $h \approx 21{,}000$ (ratio 1,348) and throughput collapses to $\sim 50{,}000$ ops/15 s versus $\sim 1.15 \times 10^6$ for uniform. Uniform keys stabilise at $h = 50$ for $n \approx 199{,}000$ (ratio 2.84, consistent with $\Theta(\log n)$ random BST height). Skewed keys yield $h = 34$–48 (ratio ~ 2.7), slightly above uniform due to left-spine load, but still $O(\log n)$. The key risk is monotone insertion order, not distribution skew per se.

5 Reflection on the Use of LLMs

Tools and models used.

- **GPT- 5.3** - Research purposes, examples and explaining papers.
- **Claude Haiku 4.5** - In VS Code Chat option for code generation
- **Claude Sonnet 4.6** - In making content for slides, and phrasing in report and answering queries.

What worked well. LLM helped out really well with **benchmarking scripts** and **python scripts** for plotting results. LLM (Claude Haiku 4.5) was able to generate both the qcheck-lin and qcheck-stm when provided the bst.mli (and AtomicFlagTag.mli) along with the intentions of those functions. I did provide the lecture 7 [2] implementation as well to use as a template and it did very well. The next_state logic etc. were correct. The only issue was it using some libraries which were discontinued/renamed to another library.
Relevent commits: - `47319dbf707a2c18cdcb7d3bebb528a27ac0b60a`
The **CAS loop** was correctly implemented by Claude Sonnet 4.6 but is was not optimized well. Since we had 3 values in the Atomic record, the generated code passed the parameters as a full record. On further research, we found that constructing two full record objects on every CAS call introduces heap allocation and GC pressure and re-implemented the method ourselves.
Relevant commits: - `eca8ce95b87424440f9e78c913399a4bb473d054`
LLMs were good in giving examples or analogies for a relatively harder part of a research paper

which helped us understand those concepts quicker. We were able to fix the data races without the use of LLMs

What did not work. LLM implemented CAS using two full records as the parameters. We found it to be mismatching with the template that we did in lecture-07 [2]. On further reading and research, we understood the current implementation causes unnecessary heap allocation and GC pressure. We implemented it ourselves later. The LLM produced incorrect API calls for certain modules (e.g., **generators** in QCheck-STM), likely due to outdated training data or mismatches with the current library APIs. These discrepancies required careful manual debugging and correction.

What was surprising. LLMs were very good at generating code given right context, references and a descriptive prompt. Web models without sufficient context tend to make mistakes and performed poorly compared to LLMs integrated in the workspace. LLM were bad at generating slides based on a fixed template even with detailed information.

What was difficult. LLMs lacked the ability to reason about low-level execution semantics and language-specific concurrency primitives (e.g., the retry behaviour of a CAS loop). Such minor mistakes required careful reading and research in-order to optimize a specific part which could have been easily missed out. LLMs were also not able to reason well whether a structural equality or a physical equality has to be used while comparing keys.

Your overall assessment. LLMs helped develop concurrent data structure implementations much faster, but required careful analysis and a solid understanding of the codebase to debug subtle issues and spot minor inefficiencies — areas where LLMs struggled to reason about low-level execution semantics. For testing and plotting scripts however, LLMs were largely self-sufficient given a detailed prompt and relevant references.

We would use LLMs the same way again, making sure we have a clear understanding of what we are asking it to generate and always verifying the output rather than taking it at face value.

6 Conclusions

The lock-free unbalanced BST can compete with — and under write-heavy workloads outperform — lock-based and optimistic baselines, but falls short of a lock-free skip list under read-dominated access patterns where the skip list's multi-level index amortises traversal cost more effectively. The lack of rebalancing is benign under uniform, normal, and skewed key distributions, where tree height stays within a small constant factor of $\log_2 n$ and throughput is largely unaffected by distribution shape. However, adversarial monotone (sorted-ascending) insertion degrades the tree to an $O(n)$ right-spine chain, causing throughput to collapse to near zero regardless of thread count — a fundamental limitation of any unbalanced BST that no amount of lock-freedom can remedy.

First, the **Natarajan-Mittal lock-free BST** outperforms the coarse-grained baseline across all workloads and thread counts, confirming that the cooperative helping mechanism and reduced contention window translate to measurable gains in a functional language runtime. Second, the optimistic lazy BST is competitive or superior under read-heavy workloads, where its zero-overhead read path dominates, but the lock-free BST overtakes it as the write fraction grows, since optimistic validation suffers frequent re-traversals under high contention. Third, key distribution shape — uniform, normal, or skewed — has negligible impact on lock-free BST throughput, provided insertion order is not monotone. Fourth, the **OCaml 5** implementation successfully demonstrates that the algorithm's core ideas are portable to a managed-runtime setting.

Limitations. The evaluation is limited to a 6-core machine with 8 hardware threads, so scalability at higher core counts remains untested. Memory reclamation is not implemented (e.g., hazard pointers [3]).

The tree is not rebalanced, and under adversarial insertion orders it degenerates into a linear chain with $O(n)$ height. Finally, the evaluation focuses on throughput only and does not account for latency or fairness under contention.

Future work. Given more time, we would extend the evaluation to a many-core machine (32+ threads) to stress-test the helping mechanism under sustained high contention. We want to add our work to ocaml-multicore/saturn repository which currently lacks an implementation for concurrent-BST if we get time. We would extend the implementation to make is self-rebalancing using a periodic rebalancing pass.

Contributions

Member	Contribution (%)	Main responsibilities
Yoogi Kovendhan	34%	lock-free implementation,evaluation plots, presentation,benchmarking
Ajoy Mathew	33%	Atomic Flag, report, lock-free implementation
Rushiraj Ralebhat	33%	Atomic Flag, QCheck-Lin, QCheck-stm, tests, slides
Total	**100%**	

References

[1] Tim Harris. A pragmatic implementation of non-blocking linked-lists. In *Proceedings of DISC*, pages 300–314, 2001. doi:10.1007/3-540-45414-4_21.

[2] Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2nd edition, 2020.

[3] Maged M. Michael. Hazard pointers: Safe memory reclamation for lock-free objects. In *IEEE Transactions on Parallel and Distributed Systems*, volume 15, pages 491–504, 2004. doi:10.1109/TPDS.2004.8.

[4] Aravind Natarajan and Neeraj Mittal. Fast concurrent lock-free binary search trees. In *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 317–328, 2014. doi:10.1145/2555243.2555256.